UNIVERSIDAD DE BURGOS

REGIONAL UNIVERSITY NETWORK
EUROPEAN UNIVERSITY

GRADO EN TECNOLOGÍAS DIGITALES PARA LA EMPRESA

Universidad de Burgos

ASIGNATURA

Programación Modular y Orientada a Objetos

TRABAJO / PRÁCTICA

Refactorización de una planta industrial mediante POO

Estudiante	Pendiente de completar
DNI / NIE	Pendiente de completar
Correo electrónico	Pendiente de completar
Grupo	Pendiente de completar
Profesor/a	Pendiente de completar
Curso académico	2026–2027
Fecha de entrega	31 de octubre de 2026

Índice

1. Introducción **3**

2. Objetivos **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Análisis del problema **4**

3.1. Requisitos funcionales 4

3.2. Restricciones y consideraciones 4

4. Diseño orientado a objetos **4**

4.1. Identificación de clases 5

4.2. Diagrama de clases UML 5

4.3. Relaciones entre clases 7

5. Principios de programación orientada a objetos aplicados **7**

5.1. Encapsulación 7

5.2. Abstracción 7

5.3. Herencia 7

5.4. Polimorfismo 8

6. Implementación **8**

6.1. Estructura del proyecto 8

6.2. Clases y métodos principales 9

6.3. Fragmentos de código relevantes 10

7. Manejo de errores y validaciones **11**

8. Pruebas y validación **11**

9. Resultados **12**

10.Discusión **12**

11.Conclusiones **13**

12.Contribución individual **13**

13.Declaración sobre el uso de herramientas de inteligencia artificial **13**

14.Referencias **13**

A. Código fuente y repositorio **15**

B. Material adicional **15**

1. Introducción

Esta práctica refactoriza una simulación funcional de una planta industrial, originalmente organizada de forma principalmente procedural, para representar sus componentes mediante objetos con responsabilidades diferenciadas. La planta simulada en MuJoCo contiene cuatro cintas que forman un circuito, una quinta cinta que conduce al horno, piezas de distintas geometrías, cámaras de inspección y un manipulador Franka Emika Panda. OpenCV se utiliza para procesar las imágenes capturadas desde las cámaras.

La solución mantiene el simulador y las rutinas físicas proporcionadas, pero añade una capa orientada a objetos para coordinar la planta, encapsular el estado de las cintas y del horno, representar las piezas y asociar cada cámara con su cinta. La cinemática inversa (IK) del Panda se reutiliza como componente existente; no se desarrolla ni se sustituye su algoritmo.

El punto de entrada es `main.py`. La lógica orientada a objetos reside principalmente en `src/`, mientras que `planta_4_cintas.py` conserva funciones auxiliares de simulación y configuración procedentes de la base del proyecto. La aplicación está escrita en Python y combina MuJoCo, NumPy y OpenCV.

2. Objetivos

2.1. Objetivo general

Reorganizar la simulación de la planta industrial mediante clases con responsabilidades claras, manteniendo su funcionamiento y preparando una arquitectura modular que pueda ampliarse en la Práctica II.

2.2. Objetivos específicos

1. Encapsular el estado y las operaciones de cada cinta transportadora.
2. Asociar cámaras de inspección con sus cintas y procesar imágenes con OpenCV.
3. Representar las piezas y sus datos relevantes para la simulación.
4. Gestionar la recepción de piezas, la temperatura y la representación visual del horno.
5. Coordinar los componentes desde `PlantaIndustrial` y conservar la integración con MuJoCo.
6. Reutilizar el controlador IK del robot Panda sin reimplementar su cinemática.

3. Análisis del problema

El código de partida ya resolvía la simulación física y el transporte de piezas, pero distribuía responsabilidades entre funciones, configuración global y estructuras compartidas. La práctica pide trasladar el estado propio de cada elemento a objetos, mantener separada la entrada del programa y lograr que la detección de una cámara pueda influir en la cinta correspondiente. También requiere que la temperatura del horno se refleje en su geometría de MuJoCo.

3.1. Requisitos funcionales

1. Ejecutar la planta y visualizar la escena mediante MuJoCo.
2. Gestionar de forma independiente cinco cintas, con controles de marcha y parada.
3. Capturar imágenes de cinco cámaras y detectar presencia y forma de piezas.
4. Actualizar la variable booleana `objeto_detectado` de cada cámara y detener la cinta asociada durante la inspección.
5. Registrar las piezas que llegan al horno, actualizar su temperatura y procesarlas al alcanzar el umbral definido.
6. Cambiar el color de la geometría del horno según su temperatura.
7. Mantener disponible el controlador Panda y su IK proporcionados.

3.2. Restricciones y consideraciones

La solución debe continuar utilizando MuJoCo para la simulación y OpenCV para la visualización y el procesamiento de cámaras. La IK del robot es código proporcionado y queda fuera del desarrollo matemático de esta práctica. Las funciones auxiliares y constantes físicas siguen centralizadas en `planta_4_cintas.py`; por tanto, la refactorización reduce el uso de estado global para los componentes modelados, pero no elimina toda la configuración procedural heredada. El enunciado requiere que el informe final incluya capturas y un enlace al repositorio GitHub; ninguno de esos elementos externos está presente en los archivos disponibles para completar esta memoria.

4. Diseño orientado a objetos

La clase coordinadora crea y mantiene las instancias de cintas, cámaras, piezas, horno y robot. Cada cámara conserva una referencia a su cinta; la planta sincroniza el estado

lógico de las piezas con los cuerpos de MuJoCo y actualiza las fuerzas, actuadores, cámaras y controles. Las rutinas físicas auxiliares siguen delegadas en el módulo preexistente.

4.1. Identificación de clases

Cuadro 1: Clases principales de la aplicación.

Clase	Responsabilidad	Elementos principales
`PlantaIndustrial`	Coordina la simulación, los componentes y la interfaz.	`inicializar()`, `actualizar()`, `ejecutar()`, controles de teclado.
`CintaTransportadora`	Mantiene el estado, la dirección y los objetos asociados a una cinta.	`iniciar()`, `detener()`, `conmutar()`, `esta_activa()`.
`CamaraInspeccion`	Captura imágenes, clasifica contornos y comunica detecciones.	`capturar()`, `detectar_forma()`, `procesar()`, `objeto_detectado`.
`ObjetoIndustrial`	Representa una pieza, su forma, segmento, posición y orientación.	`actualizar()`, `cambiar_segmento()`, `obtener_pose()`.
`Caja`, `Cilindro`	Especializan la construcción de piezas según su forma.	Heredan de `ObjetoIndustrial`.
`Horno`	Registra y procesa piezas y gestiona temperatura y color.	`registrar_pieza()`, `actualizar()`, `actualizar_color_mujoco()`.
`RobotPanda`	Adapta el controlador IK existente al coordinador de la planta.	Inicialización, control manual/automático y sincronización de pose.

4.2. Diagrama de clases UML

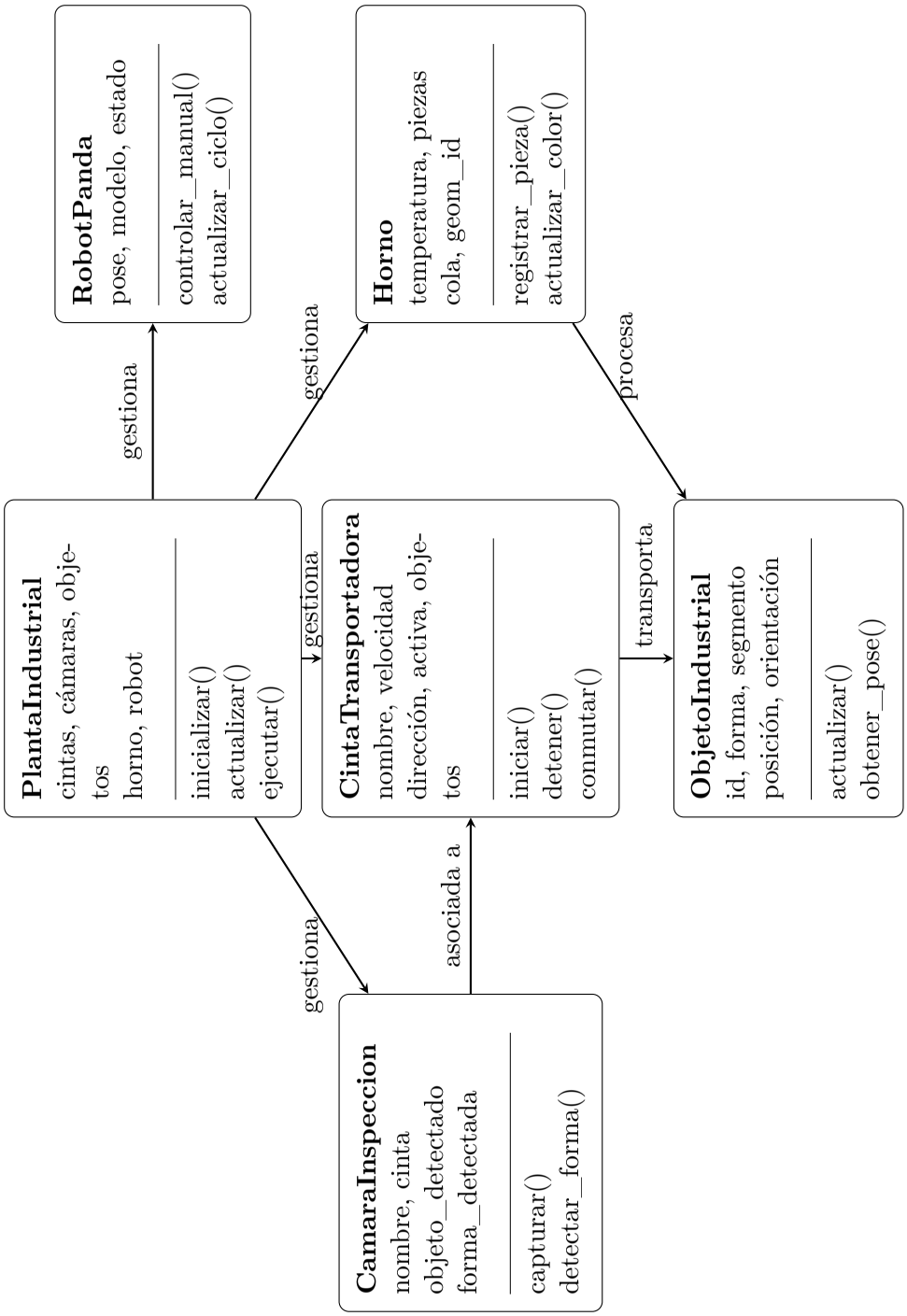


Figura 1: Diagrama de clases UML de la aplicación.

4.3. Relaciones entre clases

- **Composición y agregación:** `PlantaIndustrial` mantiene colecciones de cintas, cámaras y piezas, además del horno y el adaptador del robot. Las cámaras reciben una cinta asociada.
- **Asociación:** las cintas conservan las piezas que circulan por ellas; el horno mantiene una cola de piezas registradas.
- **Herencia:** `Caja` y `Cilindro` derivan de `ObjetoIndustrial` y fijan la forma inicial.
- **Dependencia:** `RobotPanda` delega el cálculo y control IK en el módulo `robot.py`; `PlantaIndustrial` utiliza las funciones físicas de `planta_4_cintas.py`.

5. Principios de programación orientada a objetos aplicados

5.1. Encapsulación

El estado de cada cinta (activa, velocidad, dirección y objetos) reside en su instancia y se modifica mediante métodos de marcha, parada y conmutación. La cámara mantiene sus resultados de inspección y referencia a la cinta asociada. El horno agrupa la temperatura, sus límites, el registro de piezas y el identificador de la geometría visual. La encapsulación es principalmente convencional en Python: muchos atributos son accesibles directamente; la cola del horno y algunos detalles de coordinación se indican como internos mediante el prefijo `_`.

5.2. Abstracción

Las clases expresan conceptos de la planta sin exponer al usuario de la aplicación los detalles de cada operación: por ejemplo, `detener()` no necesita conocer los actuadores de MuJoCo, y `registrar_pieza()` agrupa las actualizaciones del horno. La planta coordinadora oculta la secuencia de inicialización, actualización física y cierre de ventanas.

5.3. Herencia

`Caja` y `Cilindro` heredan de `ObjetoIndustrial`. En esta implementación, solo especializan el constructor para asignar una forma inicial y reutilizan la identidad, segmento,

posición, orientación y operaciones comunes de la clase base. `PlantaIndustrial` después puede sustituir esa forma inicial según la configuración de cada pieza. Como las subclases no añaden comportamiento propio ni sobrescriben operaciones, su papel actual es principalmente ofrecer constructores específicos y servir de posible punto de extensión, no implementar reglas distintas para cada forma.

5.4. Polimorfismo

No se define una jerarquía con sobrescritura de comportamiento ni clases abstractas. La herencia se utiliza para compartir la inicialización y el estado común de las piezas; por ello, no se atribuye a la solución un uso significativo de polimorfismo.

6. Implementación

La aplicación construye el modelo y los datos de MuJoCo, enlaza las clases con los cuerpos y actuadores de la escena, ejecuta el ciclo de simulación y libera los recursos gráficos al terminar. El ciclo de control incluye el movimiento de las piezas, la interacción con el robot, la actualización del horno y la captura periódica de cámaras.

6.1. Estructura del proyecto

- `main.py`: punto de entrada; crea `PlantaIndustrial` y arranca la simulación.
- `src/__init__.py`: declara el paquete y exporta las clases principales.
- `src/planta_industrial.py`: configura y coordina MuJoCo, las cintas, las cámaras, las piezas, el horno y el robot.
- `src/cinta_transportadora.py`: encapsula el estado, la velocidad y el movimiento de los objetos de cada cinta.
- `src/camara_inspeccion.py`: captura imágenes, detecta y clasifica formas, y comunica el estado de inspección a su cinta.
- `src/objeto_industrial.py`: define la clase base `ObjetoIndustrial` y sus subclases `Cilindro` y `Caja`, que reutilizan el estado y las operaciones comunes de las piezas.
- `src/horno.py`: gestiona la temperatura, la cola y el procesamiento de piezas, y el color de la geometría del horno en MuJoCo.
- `src/RobotPanda.py`: adapta el controlador IK existente para inicializar y controlar el Panda desde la planta.

- `planta_4_cintas.py`: conserva la configuración y las funciones procedurales de la simulación original, reutilizadas para la física, el transporte y la representación.
- `robot.py`: controlador procedural del Panda y cinemática inversa reutilizados por `src/RobotPanda.py`.
- `vendor/mujoco_menagerie/build_registry.py`: comprueba los modelos MJCF y genera el registro de modelos de Menagerie.
- `vendor/mujoco_menagerie/catalog.py`: mantiene metadatos del catálogo de robots, como nombres, vistas previas y licencias.
- `vendor/mujoco_menagerie/format_xml.py`: formatea archivos XML/MJCF.
- `vendor/mujoco_menagerie/generate_gallery.py`: genera vistas previas y actualiza la galería de Menagerie.
- `vendor/mujoco_menagerie/regenerate_license.py`: regenera el archivo de licencias de Menagerie a partir de las licencias de sus modelos.
- Los cinco scripts de `vendor/mujoco_menagerie/` son herramientas del repositorio incluido, no forman parte del arranque normal de la aplicación.
- `src/planta.xml`: define la geometría, los cuerpos y las cámaras MuJoCo de la planta.
- `requirements.txt`: recoge las dependencias Python; `README.md`: documenta instalación, ejecución y controles.
- `vendor/mujoco_menagerie/`: contiene el modelo Franka Panda y sus recursos.

6.2. Clases y métodos principales

`PlantaIndustrial` configura las cinco cintas y sus cámaras, crea las piezas, inicializa MuJoCo y el robot, y actualiza los componentes durante la ejecución. Además gestiona las teclas: `Q`/`Esc` para salir, `P` para pausar, `1`–`5` para conmutar cintas, `R` para reiniciar, `S` para guardar el mosaico y `T`/`H` para controlar la pose de prueba del robot cuando está activado el modo IK manual.

`ObjetoIndustrial` es la clase base porque todas las piezas comparten identidad, forma, segmento, posición, orientación y estado de procesamiento, así como operaciones para actualizar su posición y consultar su pose. Las subclases `Cilindro` y `Caja` reutilizan esa implementación y ofrecen constructores que fijan una forma inicial (`çilindro"` y `çubo"`, respectivamente), evitando repetir la inicialización común al crear esos tipos de pieza. En la configuración actual, `PlantaIndustrial` construye las piezas con estas subclases, pero después asigna su forma según `FORMAS`; por eso la forma lógica final puede diferir

de la forma inicial del constructor. Las subclases no añaden todavía comportamiento ni atributos propios. Esta separación resulta útil como punto de extensión si en el futuro cada tipo necesita propiedades o reglas específicas; si no se prevé esa diferencia, también bastaría con crear `ObjetoIndustrial` pasando la forma como argumento.

`CintaTransportadora` mantiene su propio estado, velocidad y dirección. `CamaraInspeccion` captura el render de la cámara MuJoCo, segmenta la imagen en HSV, busca contornos externos y clasifica el mayor contorno válido mediante circularidad y aproximación poligonal. Descarta contornos con área inferior a 90 o superior a 26000 píxeles, así como dimensiones menores de 8 píxeles. Puede devolver círculo, triángulo, cuadrado, rectángulo o desconocido.

La variable booleana `objeto_detectado` indica si el procesamiento actual encontró una forma. La planta detecta el flanco de una detección, detiene la cinta asociada y la reanuda tras 0,35 segundos de simulación, siempre que no exista una parada manual. Separar las paradas manuales del bloqueo temporal de inspección evita que la reanudación automática anule una orden manual.

`Horno` recibe piezas en una cola, mantiene los contadores y modifica su temperatura. Parte de 20 °C, con límites de 15 °C y 250 °C; su color lógico es azul por debajo de 80 °C, naranja entre 80 °C y 180 °C, y rojo desde 180 °C. Cuando hay piezas en cola, la actualización aumenta la temperatura 0,01 °C por paso y procesa una pieza al alcanzar 180 °C; sin piezas, la temperatura disminuye hacia el mínimo. El color se aplica a la geometría `oven_hot_geom` mediante `model.geom_rgba`.

`RobotPanda` envuelve las llamadas del controlador ya existente. El código de configuración mantiene activado por defecto `ROBOT_IK_TEST_MODE`, por lo que la ejecución inicial usa el control manual de la pose IK de prueba; la rutina automática de recogida y colocación queda disponible en la rama de ejecución cuando ese modo se desactiva.

6.3. Fragmentos de código relevantes

El siguiente fragmento resume el criterio de color implementado en el horno:

```
if self.temperatura < 80:
    self.color = "azul"
elif self.temperatura < 180:
    self.color = "naranja"
else:
    self.color = "rojo"
```

La implementación completa se consulta en los módulos del proyecto y deberá publicarse en el repositorio GitHub indicado en la versión final del informe.

7. Manejo de errores y validaciones

- Las clases validan dimensiones de posiciones y desplazamientos, dirección de la cinta, límites del horno e incrementos de temperatura; los valores fuera de rango producen `ValueError`.
- Se comprueba que la planta esté inicializada antes de actualizarla y que el robot esté vinculado al modelo antes de controlarlo; en caso contrario se genera `RuntimeError`.
- La inicialización comunica mediante errores explícitos la ausencia de la geometría del horno o de un actuador requerido.
- Si no se puede escribir una captura, se informa mediante `OSError`.
- Los errores no se ocultan con valores de sustitución: se propagan para que el problema pueda identificarse.

8. Pruebas y validación

No se encontraron pruebas automatizadas en los archivos disponibles. La tabla siguiente es un plan de comprobación derivado de los requisitos; sus resultados deben marcarse como realizados únicamente después de ejecutar y observar cada caso en el entorno de entrega.

Cuadro 2: Plan de pruebas funcionales pendiente de ejecución y registro.

Prueba	Entrada / escenario	Resultado esperado	Estado
P01	Iniciar `python main.py`.	Arrancan la simulación MuJoCo y la ventana OpenCV.	Pendiente
P02	Conmutar cintas con las teclas `1`–`5`.	Cada cinta cambia su estado sin conmutar las demás.	Pendiente
P03	Presentar piezas a las cámaras.	Se actualiza `objeto_detectado` y se clasifica una forma visible.	Pendiente
P04	Mantener una detección y esperar su liberación.	La cinta se detiene durante la inspección y se reanuda salvo parada manual.	Pendiente
P05	Hacer llegar una pieza al horno y observar su evolución.	Cambian temperatura y color; al alcanzar el umbral se procesa una pieza.	Pendiente
P06	Usar `T` y `H` en modo IK de prueba.	El Panda recibe las poses mediante el controlador proporcionado.	Pendiente

9. Resultados

El proyecto integra las clases descritas con el modelo físico existente y expone controles interactivos para las cintas, la pausa, el reinicio, la captura de imágenes y el movimiento de prueba del robot. El comportamiento descrito corresponde a la inspección del código; la ejecución completa de los casos de prueba queda pendiente de registrar.

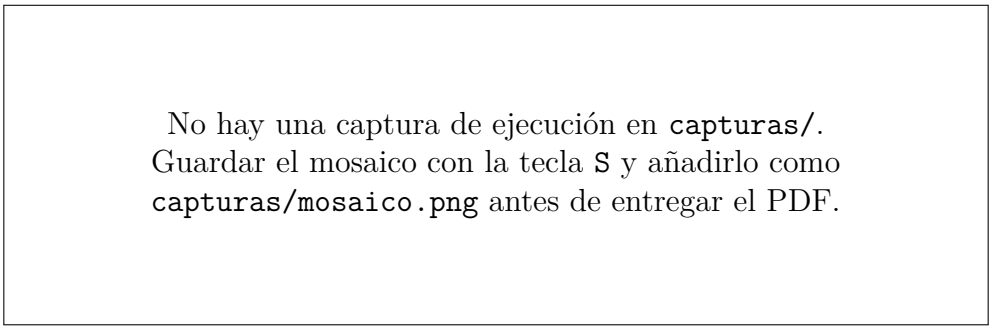


Figura 2: Mosaico de cámaras y vista general de la planta.

10. Discusión

- **Separación de responsabilidades:** las clases de dominio agrupan estado y operaciones propios, mientras la planta coordina la simulación y la interfaz.
- **Integración gradual:** se conservan las funciones físicas y el solucionador IK de partida, evitando reescribir componentes que no forman parte del objetivo de la práctica.
- **Acoplamiento:** `PlantaIndustrial` conoce el modelo MuJoCo y el módulo procedural auxiliar; esta dependencia facilita coordinar el ciclo actual, aunque limita la independencia del coordinador para pruebas unitarias.
- **Visión:** la clasificación por umbrales de HSV y geometría de contornos es sencilla y explicable, pero depende de las condiciones de iluminación y segmentación de la escena. La salida contempla formas geométricas, no una identificación semántica robusta de cada pieza.
- **Mejoras futuras:** añadir pruebas automatizadas, separar la política de decisión de la cámara, configurar los parámetros de visión por cámara y ampliar la coordinación del robot para la Práctica II.

11. Conclusiones

El proyecto estructura mediante objetos los componentes principales de una planta simulada: cintas independientes, cámaras asociadas, piezas, horno, coordinador y adaptador del robot. La cámara comunica su detección mediante `objeto_detectado`; el coordinador usa esa señal para detener temporalmente la cinta, respetando las paradas manuales. El horno encapsula su temperatura y comunica su estado térmico mediante el color de una geometría MuJoCo.

La solución mantiene el simulador y el controlador IK existentes, en lugar de reimplementar su física o cinemática. Esto proporciona una base modular para extensiones posteriores, aunque persiste acoplamiento con las rutinas procedurales originales. Para cerrar la entrega se deben ejecutar y registrar las pruebas, incorporar una captura real, completar los datos personales y añadir el enlace al repositorio.

12. Contribución individual

La información de los archivos del proyecto no permite identificar el nombre del autor ni el reparto de tareas. Completar esta sección con la contribución real de cada integrante antes de entregar. Si el trabajo es individual, indicarlo expresamente y describir las tareas realizadas por el estudiante.

13. Declaración sobre el uso de herramientas de inteligencia artificial

> Se utilizó un asistente de inteligencia artificial como apoyo para redactar esta memoria a partir del enunciado y de los archivos del proyecto. El estudiante debe revisar el contenido, comprobar que describe fielmente su trabajo y adaptarlo a las pruebas y contribuciones que haya realizado antes de entregar.

14. Referencias

Referencias

[1] Universidad de Burgos, *Práctica I: Refactorización de una Planta Industrial mediante Programación Orientada a Objetos*, enunciado de la asignatura, curso 2026–2027.

[2] Object Management Group, *Unified Modeling Language (UML)*, https://www.omg.org/spec/UML/.

[3] Google DeepMind, *MuJoCo Documentation*, https://mujoco.readthedocs.io/.

[4] OpenCV, *OpenCV-Python Tutorials*, https://docs.opencv.org/.

[5] Python Software Foundation, *Python Documentation*, https://docs.python.org/3/.

A. Código fuente y repositorio

El código debe publicarse en un repositorio GitHub accesible para la evaluación. **Enlace al repositorio:** https://github.com/martinPenalva/Practica1_PROMOD_Martin_Penalva.git.

Para instalar las dependencias desde la raíz del proyecto:

```
python -m pip install -r requirements.txt
```

Para ejecutar la aplicación:

```
python main.py
```

El modelo Franka se incluye en `vendor/mujoco_menagerie`; el README también documenta la variable `MUJOCO_MENAGERIE_PATH` para indicar otra instalación.

B. Material adicional

La carpeta `capturas/` está prevista para las imágenes guardadas durante la simulación. Incorporar aquí o en la sección de resultados la captura solicitada por el enunciado y, si procede, material adicional de las pruebas.